

NEXTERN

Internship Recommendation Engine

Hackathon Project Documentation

Project	Nextern – Internship Recommendation Engine
Purpose	Personalized internship discovery based on resume and skills
Core Features	Resume & Skills • Skill Match % • Internship Recommendations • Mock Tests
Frontend	React + Vite
Backend	FastAPI
Database	Supabase
Repository	github.com/pullabhuvanakruthi/Nextern

Nextern connects a candidate's profile with suitable internship opportunities by analyzing skills, presenting an understandable match percentage, and supporting preparation through mock tests.

Table of Contents

- 1. Project Overview
- 2. Problem Statement
- 3. Objectives
- 4. Scope and Key Features
- 5. Technology Stack
- 6. System Architecture
- 7. Application Workflow
- 8. Resume and Skills Processing
- 9. Skill Match Percentage
- 10. Internship Recommendation Module
- 11. Mock Test Module
- 12. Frontend Design and Navigation
- 13. Backend and API Layer
- 14. Supabase Database Design
- 15. Data Flow
- 16. Security and Privacy
- 17. Validation and Error Handling
- 18. Testing Strategy
- 19. Performance and Scalability
- 20. Deployment and Configuration
- 21. User Journey and Demo Flow
- 22. Limitations and Future Enhancements
- 23. Hackathon Value Proposition
- 24. Conclusion
- Appendix A. API Contract
- Appendix B. Database Entities

1. Project Overview

Nextern is an internship recommendation engine designed to help students discover internship opportunities that align with their current skills and resume profile. The system accepts candidate information, organizes relevant skills, compares them with internship requirements, and presents suitable opportunities.

The platform combines recommendation with preparation. A candidate can understand how closely their skills align with a role through a **Skill Match Percentage** and use **Mock Tests** to assess technical readiness.

The application uses React with Vite for the frontend, FastAPI for the backend, and Supabase for database services. This provides a clean full-stack structure suitable for a web-based internship recommendation platform.

2. Problem Statement

Students often spend considerable time searching through internship listings without knowing which opportunities are genuinely relevant to their skills. Comparing a resume with individual role requirements manually can be difficult, especially when a candidate has skills across multiple technologies.

Another challenge is readiness. Finding an internship does not automatically tell a student whether they have enough of the required skills or what they should practice before applying.

Nextern addresses these challenges through a single workflow that connects **candidate profile** → **skill matching** → **internship recommendation** → **skill preparation**.

3. Objectives

- Accept candidate resume and/or manually entered skills.
- Build a structured candidate skill profile.
- Compare candidate skills with internship requirements.
- Calculate and display a clear Skill Match Percentage.
- Recommend internships that are relevant to the candidate.
- Provide mock tests for technical preparation.
- Present personalized recommendations and assessment results through a web interface.
- Store relevant application information using Supabase.
- Provide an extensible foundation for improved recommendation and assessment capabilities.

4. Scope and Key Features

Feature	Purpose	Input	Output
Resume / Skill Input	Create candidate profile	Resume, skills	Candidate skill profile
Skill Match %	Measure role alignment	Candidate and role skills	Match percentage
Internship Recommendation	Find suitable opportunities	Candidate profile	Recommended internships
Mock Tests	Assess technical readiness	Skill or topic	Questions, score, result
Dashboard	Present personalized results	Profile and results	Recommendations and progress
Database	Persist application information	Profiles, roles, results	Stored records

5. Technology Stack

Layer	Technology	Purpose
Frontend	React	Component-based user interface
Build Tool	Vite	Development server and production build
Backend	FastAPI	API layer and application logic
Database	Supabase	Persistent application data
Frontend Language	JavaScript / JSX	Client-side implementation
Backend Language	Python	Server-side implementation
Version Control	Git / GitHub	Source-code management

The selected stack supports rapid development, clear separation of responsibilities, and straightforward extension of the recommendation and assessment modules.

6. System Architecture

Nextern follows a layered client-server architecture. The React/Vite frontend handles user interaction and presentation. FastAPI provides the backend API and application logic. The recommendation and matching logic operates through the backend, while Supabase provides persistent data storage.

High-Level Architecture

User → React/Vite → FastAPI → Matching / Recommendation / Mock-Test Logic → Supabase

Component	Responsibility
React/Vite Client	Resume and skill input, dashboards, internship results, match indicators, mock-test screens and result presentation
FastAPI Backend	Request validation, profile processing, matching, recommendation and mock-test operations
Matching / Recommendation Logic	Compare candidate skills with internship requirements and calculate relevance
Supabase	Persist candidate, skill, internship, test and result information

7. Application Workflow

Step	Action	Result
1	Open Nextern	Application interface
2	Enter or upload resume and skills	Candidate information
3	Process relevant skills	Structured skill profile
4	Compare skills with internship requirements	Skill Match Percentage
5	Rank suitable opportunities	Internship recommendations
6	Open a recommendation	Role and skill alignment
7	Attempt a mock test	Evaluated responses
8	View result	Score and preparation feedback

The central user journey is **Profile** → **Analyze** → **Match** → **Recommend** → **Practice** → **Improve**.

8. Resume and Skills Processing

The resume and skills stage provides the information required to personalize recommendations. A resume may contain technologies, programming languages, frameworks, projects and other information. The recommendation workflow focuses on relevant skills that can be compared against internship requirements.

The processing flow is:

- 1 Receive resume content and/or manually entered skills.
- 2 Identify relevant technical skills.
- 3 Normalize skill names where appropriate.
- 4 Remove duplicate skill entries.
- 5 Associate the resulting skills with the candidate profile.
- 6 Use the normalized skill set for matching and recommendation.

Manual skill entry can complement resume-derived information and allows the candidate to control the skills considered by the recommendation engine.

9. Skill Match Percentage

Skill Match Percentage is the primary indicator used to communicate how closely a candidate's current skills align with an internship.

Baseline Calculation

$$\text{Match \%} = (\text{Matched Required Skills} / \text{Total Required Skills}) \times 100$$

For example, if an internship requires five skills and the candidate matches three, the baseline match percentage is 60%.

The scoring model can be extended to weighted requirements so that essential skills contribute more strongly than optional skills. For explainability, the interface should show the percentage together with matched and missing skills whenever the underlying data supports it.

10. Internship Recommendation Module

The recommendation module converts the candidate profile into a relevant shortlist of internships. Each opportunity is evaluated based on the relationship between its requirements and the candidate's available skills.

- 1 Load the candidate skill profile.
- 2 Retrieve internship opportunities and their requirements.
- 3 Compare candidate skills with each opportunity.
- 4 Calculate the match score.
- 5 Rank opportunities by relevance.
- 6 Return the highest-relevance opportunities to the frontend.
- 7 Display the opportunity and its match information.

Recommendation results should make the role, opportunity information and skill alignment easy to understand so that the candidate can decide which internships to explore further.

11. Mock Test Module

Mock Tests provide a preparation layer after internship discovery. Candidates can practice technical skills and evaluate their readiness through a question-based assessment.

- Select a skill or test topic.
- Display questions and answer options.
- Record candidate responses.
- Evaluate the submission.
- Calculate and display the score.

- Store results where persistence is enabled.
- Use the result as preparation feedback.

The mock-test module complements skill matching by helping candidates convert identified skill gaps into concrete preparation activities.

12. Frontend Design and Navigation

The frontend is implemented using React and Vite. The interface is organized around the candidate's journey from profile creation to recommendation and assessment.

View	Purpose	Main Elements
Landing / Entry	Introduce the platform	Project overview and action
Profile / Input	Collect candidate information	Resume and skill controls
Dashboard	Present personalized information	Match summary and recommendations
Internship Results	Explore suitable opportunities	Recommendation cards and match %
Mock Test	Practice relevant skills	Questions and answer controls
Test Result	Review performance	Score and result summary

Reusable React components can be used for skill tags, recommendation cards, match indicators, form controls, questions and result displays.

13. Backend and API Layer

FastAPI provides the backend API between the React application, recommendation logic and Supabase. It handles request validation, application processing, database interaction and structured responses.

API Area	Purpose
Profile / Skills	Create or update candidate information and skills
Resume	Receive and process resume information
Internships	Retrieve internship opportunities
Recommendations	Generate candidate-specific recommendations and match values
Mock Tests	Retrieve tests/questions and process attempts
Results	Retrieve assessment results and related progress

The backend layer keeps business logic separate from the user interface and provides a consistent interface for the platform.

14. Supabase Database Design

Supabase provides persistent storage for the core information required by the recommendation and assessment workflow.

Entity	Purpose
User / Profile	Candidate identity and profile information
Skills	Reusable skill information
Profile Skills	Relationship between candidates and skills
Internships	Internship opportunity information
Internship Skills	Skills associated with each internship

Entity	Purpose
Mock Tests	Assessment definitions
Questions	Questions associated with mock tests
Attempts / Results	Candidate assessment history and scores

These entities support the core relationships between candidates, skills, internship requirements and assessment outcomes.

15. Data Flow

Candidate Profile Flow

Resume / Skills → React/Vite → FastAPI → Skill Processing → Candidate Profile → Supabase

Recommendation Flow

Candidate Skills + Internship Requirements → FastAPI Matching → Match Scores → Ranked Recommendations → React

Assessment Flow

Skill/Topic → Mock Test → Questions → Responses → Evaluation → Score → Result

The data flow separates user interaction, business logic and persistence, making the system easier to maintain and extend.

16. Security and Privacy

Resume information may contain personal information, so candidate data should be protected throughout the application.

- Keep private database credentials and service keys on the backend.
- Store secrets and environment-specific configuration outside source code.
- Validate uploaded files and API input.
- Apply authorization before accessing user-specific records.
- Return only required information through APIs.
- Use HTTPS in deployed environments.
- Apply appropriate Supabase access policies and row-level security where applicable.
- Do not commit credentials or secrets to the repository.

The application should follow least-privilege principles so that each component receives only the access required for its function.

17. Validation and Error Handling

Input validation should occur at the API boundary before processing or database operations.

Scenario	Expected Behavior
No resume or skills	Request usable candidate information
Invalid file	Reject safely and display an understandable error
No skills extracted	Allow manual skill input
No suitable internships	Display a useful empty state
Invalid test submission	Reject the invalid request

Scenario	Expected Behavior
Database or API failure	Display safe user-facing error information

18. Testing Strategy

Testing focuses on the core recommendation and assessment workflow.

Test Level	Key Tests
Unit	Skill normalization, match calculation, score calculation and ranking
API / Integration	Profile, internship, recommendation, mock-test and database operations
UI	Input, recommendation cards, match percentage, mock-test and result screens
End-to-End	Complete profile-to-recommendation-to-test workflow
Negative Testing	Missing input, invalid data, empty results and backend failures

A successful demonstration should show profile input, recommendation generation, a meaningful match percentage and completion of a mock test.

19. Performance and Scalability

For the MVP, performance should focus on responsive recommendation requests, efficient database access and smooth frontend rendering.

- Limit recommendation queries to required fields.
- Avoid repeated database requests for the same information.
- Limit or paginate large recommendation lists.
- Return compact API responses.
- Use asynchronous backend operations for I/O-bound work where appropriate.
- Add database indexes as data volume increases.
- Introduce caching as usage grows.

The architecture can evolve toward larger-scale usage by optimizing data access, separating compute-heavy recommendation processes and adding caching.

20. Deployment and Configuration

The application can be deployed as a React/Vite frontend connected to a FastAPI backend, with Supabase providing database services.

Configuration	Purpose
Frontend API URL	Connect the React application to FastAPI
Supabase URL	Identify the Supabase project
Public client key	Client-side access under configured policies
Private/service key	Backend-only privileged operations
Application configuration	Backend environment-specific settings

Private keys and service credentials must never be included in the browser bundle or committed to public source code.

21. User Journey and Hackathon Demo Flow

- 1 Introduce Nextern and the internship discovery problem.
- 2 Enter or upload a sample resume and skills.
- 3 Display the candidate skill profile.
- 4 Open internship recommendations.
- 5 Highlight the Skill Match Percentage.
- 6 Show matched and missing skills where available.
- 7 Open a relevant mock test.
- 8 Complete the test and display the result.
- 9 Explain how the result guides further preparation.

This flow demonstrates the product value from **finding an opportunity** to **understanding the fit** and finally **preparing for the role**.

22. Limitations and Future Enhancements

Future versions of Nextern can improve personalization, recommendation quality and preparation support through:

- Semantic resume and skill extraction.
- Weighted skill matching.
- Project and experience-based matching.
- Personalized ranking using candidate feedback.
- Automatic mapping of skill gaps to mock tests.
- Adaptive mock-test difficulty.
- Internship eligibility and additional filtering.
- Resume improvement suggestions.
- Explainable recommendation reasons.
- Skill-gap analytics.
- Advanced ML/NLP-based recommendation models.

23. Hackathon Value Proposition

Problem: Students need a faster and more reliable way to identify internships that match their existing capabilities.

Solution: Nextern analyzes a candidate's skills, calculates role alignment and recommends suitable internships.

Explainability: Skill Match % gives users a clear indication of alignment.

Actionability: Mock tests help candidates prepare for relevant technical skills.

Technical Strength: React/Vite, FastAPI and Supabase are integrated into a complete full-stack workflow.

Future Potential: The platform can evolve into a personalized career preparation system using richer skill and recommendation intelligence.

24. Conclusion

Nextern is an internship recommendation engine designed to simplify internship discovery and help students understand their readiness for available opportunities.

Its core workflow combines resume and skill input, skill matching, internship recommendation and mock testing. React + Vite provides the user interface, FastAPI provides the backend application layer, and Supabase provides persistent storage.

By connecting recommendation with preparation, Nextern provides a complete journey from **candidate profile** → **internship opportunity** → **skill improvement**.

Appendix A. API Contract

Operation	Method	Purpose
Candidate Profile	POST / PUT	Create or update candidate information and skills
Resume Processing	POST	Submit and process resume information
Internships	GET	Retrieve internship opportunities
Recommendations	GET / POST	Generate recommendations and match values
Internship Details	GET	Retrieve selected internship information
Mock Test	GET	Retrieve test and question information
Test Submission	POST	Evaluate candidate responses
Test Results	GET	Retrieve assessment results

The API contract documents the functional responsibilities of the backend services.

Appendix B. Database Entities

Entity	Purpose
Profiles	Candidate profile information
Skills	Skill vocabulary and normalized skills
Profile Skills	Candidate-to-skill relationship
Internships	Internship opportunity information
Internship Skills	Internship-to-skill relationship
Mock Tests	Assessment definitions
Questions	Mock-test questions
Attempts / Results	Candidate test attempts and scores